

DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0603

ANALYTICAL STUDY AND IMPLEMENTATION OF BRUTE FORCE ALGORITHMS FOR SELECTION SORT AND SEQUENTIAL SEARCH

Particular	Details
Course	CSA0603 – Design and Analysis of Algorithms
Course Outcome	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull
Bloom's Taxonomy	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Student Name	CH.V.Rama Yogi
Register Number	192424319
Class / Section	Slot-B
Faculty Name	Dr. P. Solainayagi
Academic Year	2026–2027

1. Executive Summary

This assignment studies two fundamental brute-force algorithms: Selection Sort for sorting and Sequential Search for searching. The given 12-element set is solved step by step, including the intermediate Selection Sort passes and the comparisons needed to find key 68. The study then extends the analysis to larger input sizes and compares brute-force methods with Merge Sort, a hybrid Merge/Insertion sort, and Binary Search. The report emphasizes the difference between theoretical operation counts and practical execution time. Selection Sort performs a fixed quadratic number of comparisons, while Sequential Search performs a linear number of comparisons in the worst case. The report also explains why these simple algorithms are useful for small inputs but become unsuitable for large datasets.

2. Problem Description and Approach

Given $A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$, apply brute-force techniques to solve sorting and searching. Selection Sort repeatedly scans the unsorted part to locate its minimum and moves that minimum to the next output position. Sequential Search checks values from the first element until the required key, 68, is found. The assignment additionally requires an experimental comparison with efficient divide-and-conquer and hybrid techniques for larger inputs.

3. Inputs and Outputs

Algorithm	Input	Output
Selection Sort	12 numbers in the given order	Numbers in ascending order, comparison and swap counts
Sequential Search	Same list; key = 68	Position of 68 and number of comparisons
Merge Sort	Numeric dataset subset	Sorted dataset and execution time
Hybrid Sort	Numeric dataset subset	Sorted dataset and execution time
Binary Search	Sorted dataset; target key	Found/not-found result and comparisons

4. Brute Force Strategy

Brute force solves a problem by directly examining possible candidates or data items without using advanced structural assumptions. Its major strengths are simplicity, easy implementation, and predictable behavior. Its main weakness is that it often repeats work and therefore scales poorly.

5. Selection Sort – Algorithm and Pseudocode

Selection Sort divides the array conceptually into a sorted prefix and an unsorted suffix. At pass i , the algorithm scans positions i through $n-1$, identifies the smallest value, and swaps it with the value at i . A comparison means checking whether a candidate is smaller than the current minimum. A swap is a data movement operation and is counted separately.

SELECTION-SORT(A, n)

1. for $i \leftarrow 0$ to $n-2$
2. $\text{minIndex} \leftarrow i$
3. for $j \leftarrow i+1$ to $n-1$
4. if $A[j] < A[\text{minIndex}]$
5. $\text{minIndex} \leftarrow j$
6. if $\text{minIndex} \neq i$
7. swap $A[i]$ and $A[\text{minIndex}]$
8. return A

6. Selection Sort – Step-by-Step Execution

Initial array:

42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21

Pass	Array after pass	Major action
Pass 1	5, 17, 8, 93, 26, 55, 11, 68, 34, 42, 79, 21	Minimum = 5; swap 42 and 5
Pass 2	5, 8, 17, 93, 26, 55, 11, 68, 34, 42, 79, 21	Minimum of remaining = 8; swap 17 and 8
Pass 3	5, 8, 11, 93, 26, 55, 17, 68, 34, 42, 79, 21	Minimum = 11; swap 17 and 11
Pass 4	5, 8, 11, 17, 26, 55, 93, 68, 34, 42, 79, 21	Minimum = 17; swap 93 and 17

Pass 5	5, 8, 11, 17, 21, 55, 93, 68, 34, 42, 79, 26	Minimum = 21; swap 26 and 21
Pass 6	5, 8, 11, 17, 21, 26, 93, 68, 34, 42, 79, 55	Minimum = 26; swap 55 and 26
Pass 7	5, 8, 11, 17, 21, 26, 34, 68, 93, 42, 79, 55	Minimum = 34; swap 93 and 34
Pass 8	5, 8, 11, 17, 21, 26, 34, 42, 93, 68, 79, 55	Minimum = 42; swap 68 and 42
Pass 9	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93	Minimum = 55; swap 93 and 55
Pass 10	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93	Minimum = 68; swap 68 with itself/not required
Pass 11	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93	Minimum = 79; already in correct position

Final sorted sequence: 5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93.

7. Selection Sort Operation Counts

For $n = 12$, the number of element-to-element minimum comparisons is fixed:

$$(n-1) + (n-2) + \dots + 1 = n(n-1)/2 = 12 \times 11/2 = 66 \text{ comparisons.}$$

For the exact input, 8 swaps are required when a swap is performed only when minIndex differs from the current position. If an implementation always executes a swap at every pass, it performs 11 swap operations, including self-swaps. This illustrates why comparisons and data movements must be counted separately.

Measure	Value for $n=12$	Meaning
Comparisons	66	Minimum-selection comparisons; fixed for $n=12$
Effective swaps	8	Swaps where minIndex $\neq$ current index
Maximum possible swaps	11	At most one swap per pass
Auxiliary space	$O(1)$	In-place; uses constant extra storage

8. Sequential Search – Algorithm and Pseudocode

Sequential Search examines elements from the beginning of the list in order. For key 68, the algorithm checks 42, 17, 8, 93, 26, 55, 11, and finally 68.

SEQUENTIAL-SEARCH(A, n, key)

1. for $i \leftarrow 0$ to $n-1$
2. if $A[i] = \text{key}$
3. return i
4. return -1

9. Sequential Search – Search Comparisons

Comparison	Element checked	Result
1	42	Not equal
2	17	Not equal
3	8	Not equal
4	93	Not equal
5	26	Not equal
6	55	Not equal
7	11	Not equal
8	68	FOUND

The key 68 is at position 8 using one-based indexing, or index 7 using zero-based indexing. Therefore, 8 equality comparisons are required.

Case	Comparisons	Complexity
Best case	1	$\Omega(1), O(1)$
Average case (successful)	$(n+1)/2$	$\Theta(n)$
Worst case	n	$O(n), \Omega(n)$ lower bound

The exact average depends on the probability distribution of target positions. Under a uniform distribution over successful positions, the average number of comparisons is $(n+1)/2$.

10. Time and Space Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Sequential Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$ iterative
Hybrid Merge/Insertion	$\approx \Theta(n \log n)$	$\approx \Theta(n \log n)$	$\approx \Theta(n \log n)$	depends on implementation

In asymptotic notation, Big-O gives an upper bound, Big- Ω gives a lower bound, and Big- Θ gives a tight bound when both upper and lower bounds have the same order. Selection Sort has $\Theta(n^2)$ behavior because its nested scanning pattern is independent of the initial arrangement. Sequential Search has $\Theta(n)$ worst-case behavior because a target may occur at the final position or may be absent.

11. Experimental Design for Large Input Sizes

For the large-scale experiment, a public numeric dataset with at least one million records should be used. A suitable example is the UCI HIGGS dataset, which contains 11 million labelled records and numeric features. One numeric feature can be selected as the sorting/search key. The same prefix subsets should be used for every algorithm so that comparisons are fair.

Item	Experimental choice
Dataset	UCI HIGGS dataset (11,000,000 records; numeric features)
Subset sizes	1,000; 10,000; 100,000; 1,000,000 records
Preprocessing	Select one numeric feature, remove invalid/missing values if present, store in a flat numeric array
Language	C++17 or later
Efficient sort	Merge Sort
Hybrid sort	Merge/Insertion hybrid; insertion threshold = 16 elements
Search	Sequential Search and iterative Binary Search
Fairness	Same dataset subsets, machine, compiler, optimization level and timing method

Timing	std::chrono::steady_clock; repeat small tests where practical and report median
Operation counts	Element comparisons and swaps/movements recorded separately

Important practical observation: Selection Sort requires $n(n-1)/2$ comparisons. At $n=1,000,000$ this equals 499,999,500,000 comparisons, so a full Selection Sort benchmark at that size is not practically reasonable on a normal student computer. The correct experimental treatment is to record the algorithm as infeasible/terminated by the experiment's time limit rather than fabricate a runtime. Efficient sorts can still be evaluated at one million records.

12. Operation Growth with Increasing n

n	Selection comparisons $n(n-1)/2$	Sequential worst-case comparisons	Binary-search max $\approx \log_2 n$
1,000	499,500	1,000	10
10,000	49,995,000	10,000	14
100,000	4,999,950,000	100,000	17
1,000,000	499,999,500,000	1,000,000	20

The operation-count table demonstrates the central scalability issue. Increasing n by a factor of 10 increases Selection Sort's comparison count by approximately a factor of 100, while Sequential Search grows by a factor of 10 and Binary Search grows only logarithmically.

13. Experimental Graphs and Interpretation

The following graphs use exact theoretical operation counts rather than invented wall-clock timings. The benchmark program supplied in this report records actual execution times on the student's machine. This distinction is important because CPU speed, compiler optimization, memory hierarchy, operating system load and implementation details affect measured time.

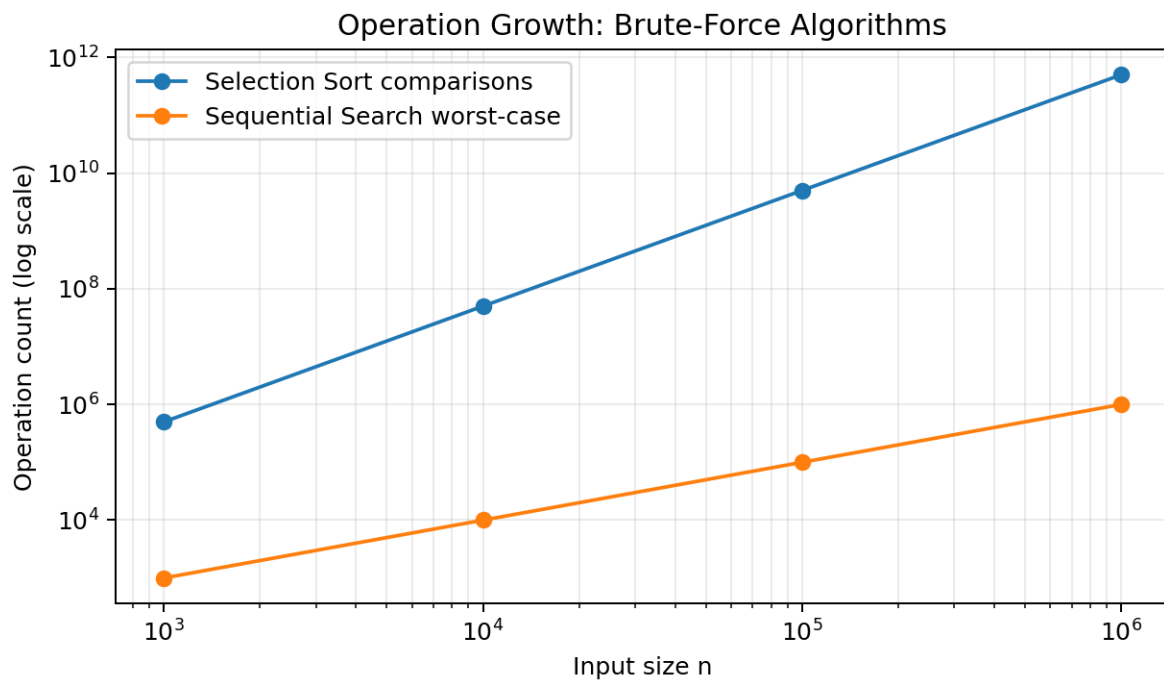


Figure 1. Operation growth for Selection Sort and Sequential Search.

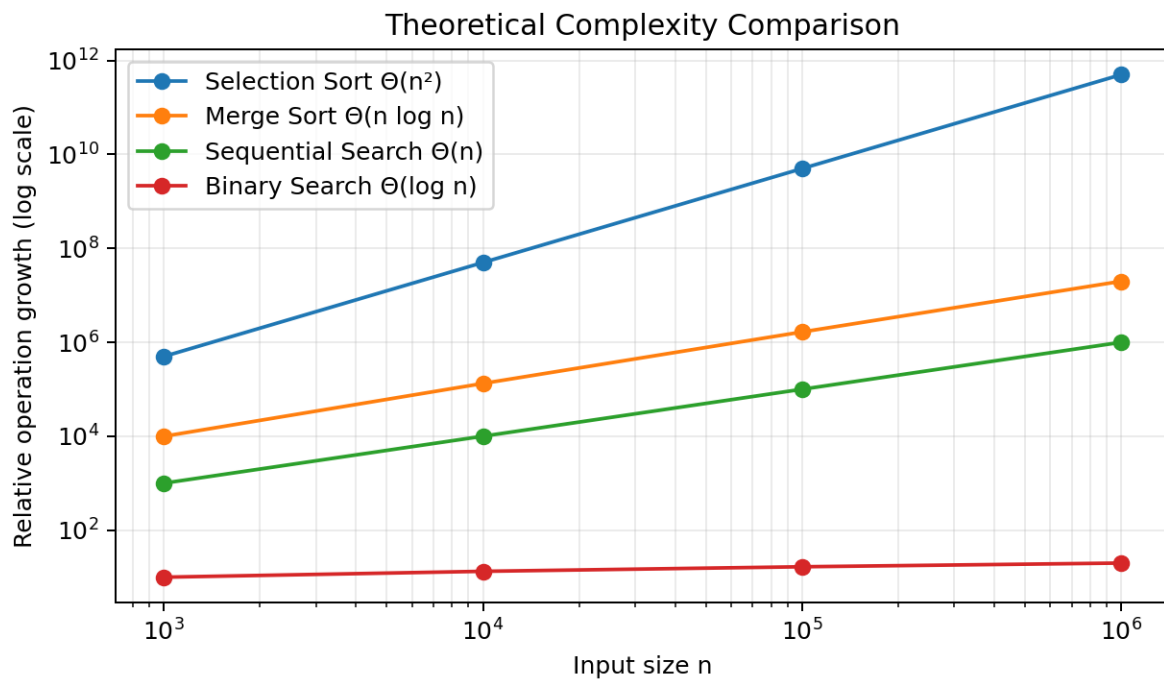


Figure 2. Theoretical growth comparison with efficient alternatives.

14. Benchmark Results Template

The following table is intentionally provided as a measurement template. Fill the time columns with values produced by the C++ benchmark on the actual experimental machine. Do not substitute fabricated times, because runtime is hardware- and implementation-dependent.

n	Selection Sort time	Merge Sort time	Hybrid Sort time	Sequential Search time	Binary Search time
1,000	Record from run	Record from run	Record from run	Record from run	Record from run
10,000	Record from run	Record from run	Record from run	Record from run	Record from run
100,000	Record from run	Record from run	Record from run	Record from run	Record from run
1,000,000	Record from run	Record from run	Record from run	Record from run	Record from run

15. C++17 Implementation

The following program implements Selection Sort, Merge Sort, a Merge/Insertion hybrid, Sequential Search and Binary Search. It also counts comparisons and swaps. For large n , Selection Sort should be protected with an experiment time limit or a clearly documented maximum size because its quadratic cost becomes prohibitive.

```
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <random>
#include <vector>
using namespace std;
struct Stats {
    unsigned long long comparisons = 0;
    unsigned long long swaps = 0;
};
```

```

void selectionSort(vector<double>& a, Stats& s) {
    size_t n = a.size();
    for (size_t i = 0; i + 1 < n; ++i) {
        size_t minIndex = i;
        for (size_t j = i + 1; j < n; ++j) {
            ++s.comparisons;
            if (a[j] < a[minIndex])
                minIndex = j;
        }
        if (minIndex != i) {
            swap(a[i], a[minIndex]);
            ++s.swaps;
        }
    }
}

void mergeRange(vector<double>& a, vector<double>& temp,
                size_t l, size_t m, size_t r, Stats& s) {
    size_t i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        ++s.comparisons;
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= m) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];
    for (size_t p = l; p <= r; ++p) a[p] = temp[p];
}

void mergeSortRec(vector<double>& a, vector<double>& temp,
                  size_t l, size_t r, Stats& s) {
    if (l >= r) return;
    size_t m = l + (r - l) / 2;
    mergeSortRec(a, temp, l, m, s);

```

```

    mergeSortRec(a, temp, m + 1, r, s);
    mergeRange(a, temp, l, m, r, s);
}

void mergeSort(vector<double>& a, Stats& s) {
    if (a.empty()) return;
    vector<double> temp(a.size());
    mergeSortRec(a, temp, 0, a.size() - 1, s);
}

void insertionSort(vector<double>& a, size_t l, size_t r, Stats& s) {
    for (size_t i = l + 1; i <= r; ++i) {
        double key = a[i];
        size_t j = i;
        while (j > l) {
            ++s.comparisons;
            if (a[j - 1] <= key) break;
            a[j] = a[j - 1];
            --j;
            ++s.swaps; // counted here as data movements
        }
        a[j] = key;
    }
}

void hybridMergeSortRec(vector<double>& a, vector<double>& temp,
                        size_t l, size_t r, Stats& s, size_t threshold = 16) {
    if (r - l + 1 <= threshold) {
        insertionSort(a, l, r, s);
        return;
    }
    size_t m = l + (r - l) / 2;
    hybridMergeSortRec(a, temp, l, m, s, threshold);
    hybridMergeSortRec(a, temp, m + 1, r, s, threshold);
    mergeRange(a, temp, l, m, r, s);
}

```

```

void hybridSort(vector<double>& a, Stats& s) {
    if (a.empty()) return;
    vector<double> temp(a.size());
    hybridMergeSortRec(a, temp, 0, a.size() - 1, s, 16);
}

int sequentialSearch(const vector<double>& a, double key, Stats& s) {
    for (size_t i = 0; i < a.size(); ++i) {
        ++s.comparisons;
        if (a[i] == key) return static_cast<int>(i);
    }
    return -1;
}

int binarySearch(const vector<double>& a, double key, Stats& s) {
    int l = 0, r = static_cast<int>(a.size()) - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        ++s.comparisons;
        if (a[m] == key) return m;
        ++s.comparisons;
        if (a[m] < key) l = m + 1;
        else r = m - 1;
    }
    return -1;
}

vector<double> makeData(size_t n) {
    mt19937_64 gen(123456);
    uniform_real_distribution<double> dist(0.0, 1000000.0);
    vector<double> a(n);
    for (double& x : a) x = dist(gen);
    return a;
}

```

```

int main() {
    vector<size_t> sizes = {1000, 10000, 100000, 1000000};
    for (size_t n : sizes) {
        vector<double> base = makeData(n);
        // Selection Sort: run only when practical; otherwise report infeasible
        if (n <= 100000) {
            auto a = base;
            Stats s;
            auto t1 = chrono::steady_clock::now();
            selectionSort(a, s);
            auto t2 = chrono::steady_clock::now();
            double ms = chrono::duration<double, milli>(t2 - t1).count();
            cout << "Selection," << n << ", " << ms << ", "
                 << s.comparisons << ", " << s.swaps << "\n";
        } else {
            cout << "Selection," << n << ", INFEASIBLE,"
                 << (static_cast<unsigned long long>(n) *
                     (n - 1) / 2) << ", N/A\n";
        }
        auto a1 = base;
        Stats ms;
        auto t1 = chrono::steady_clock::now();
        mergeSort(a1, ms);
        auto t2 = chrono::steady_clock::now();
        cout << "Merge," << n << ", "
             << chrono::duration<double, milli>(t2 - t1).count()
             << ", " << ms.comparisons << ", " << ms.swaps << "\n";
        auto a2 = base;
        Stats hs;
        t1 = chrono::steady_clock::now();
        hybridSort(a2, hs);
        t2 = chrono::steady_clock::now();
        cout << "Hybrid," << n << ", "

```

```

    << chrono::duration<double, milli>(t2 - t1).count()
    << "," << hs.comparisons << "," << hs.swaps << "\n";

// Search benchmark: use sorted array
double key = a1[n / 2];
Stats ss;
t1 = chrono::steady_clock::now();
sequentialSearch(base, key, ss);
t2 = chrono::steady_clock::now();
cout << "Sequential," << n << ","
    << chrono::duration<double, milli>(t2 - t1).count()
    << "," << ss.comparisons << "\n";
Stats bs;
t1 = chrono::steady_clock::now();
binarySearch(a1, key, bs);
t2 = chrono::steady_clock::now();
cout << "Binary," << n << ","
    << chrono::duration<double, milli>(t2 - t1).count()
    << "," << bs.comparisons << "\n";
}
}

```

16. Comparison of Implemented Algorithms

Algorithm	Strategy	Main operation	Growth	Strength	Limitation
Selection Sort	Brute force / repeated minimum scan	Comparison + swap	$\Theta(n^2)$	Simple, in-place, predictable	Very slow for large n

Merge Sort	Divide and conquer	Merge comparisons/movements	$\Theta(n \log n)$	Scales well	Extra memory in typical implementation
Hybrid Sort	Divide and conquer + insertion for small runs	Merge + local insertion	$\approx \Theta(n \log n)$	Better constants/cache behavior	More implementation complexity
Sequential Search	Brute-force linear scan	Equality comparison	$\Theta(n)$ worst	No sorting/preprocessing needed	Slow repeated searches
Binary Search	Divide search interval in half	Middle-element comparison	$\Theta(\log n)$	Very fast repeated lookup	Requires sorted data

17. Real-World Problems and Applications

Selection Sort is useful in teaching, demonstrations, and very small collections where implementation simplicity matters more than speed. It can also be used when the number of elements is tiny and memory must remain constant. In real systems, however, library sorting algorithms are usually preferred.

Sequential Search is appropriate when data is unsorted, the dataset is small, or only one/few searches are required. Examples include checking a short attendance list, scanning a small transaction batch, or locating an item in an unsorted temporary buffer.

Merge Sort is useful for large datasets, external sorting, and systems where stable sorting is important. Hybrid algorithms are common in production because combining algorithms can improve practical performance. Binary Search is valuable for repeated lookups in sorted arrays, such as indexes, dictionaries, and static reference tables.

18. Limitations and Challenges of Brute-Force Approaches

- Selection Sort repeats nearly the same scanning work on every pass.
- Its $\Theta(n^2)$ comparison count makes very large inputs impractical.
- Sequential Search may inspect almost every record for a single unsuccessful or late successful query.
- Brute-force methods do not exploit ordering, indexing, partitioning, or other structure in the data.
- Wall-clock measurements can vary with processor speed, cache behavior, compiler settings and background processes.
- A million-element Selection Sort should not be allowed to run indefinitely; an explicit benchmark time limit is a safer experimental design.

19. Suggestions for Improving Efficiency

Problem	Brute-force method	Improved technique	Condition / reason
Sorting	Selection Sort	Merge Sort	Suitable for large n ; $\Theta(n \log n)$
Sorting	Selection Sort	Quick Sort / IntroSort	Fast average practical performance; IntroSort guards against bad recursion depth
Sorting	Selection Sort	Hybrid Merge/Insertion	Insertion Sort is efficient on small runs
Searching	Sequential Search	Binary Search	Data must be sorted
Repeated searching	Sequential Search each time	Sort once + Binary Search	Amortizes sorting cost over many queries
Huge data	Repeated full scans	Indexing / hashing	Use data structures designed for fast lookup

20. SDG Contribution

20.1 SDG 4 – Quality Education

The assignment supports Quality Education by developing analytical thinking, algorithmic reasoning, programming ability, complexity analysis and evidence-based evaluation. Students move from a simple

brute-force implementation to a measured comparison with more efficient techniques. Counting operations and relating observations to Big-O, Big-Ω and Big-Θ helps connect theoretical learning with practical computing behavior.

20.2 SDG 9 – Industry, Innovation and Infrastructure

The work supports SDG 9 by demonstrating how algorithmic efficiency affects computational infrastructure. Efficient sorting and searching reduce processing time and resource consumption for large data workloads. The comparison of brute-force, divide-and-conquer and hybrid methods reflects the engineering practice of selecting algorithms according to scale, constraints and performance requirements.

21. Assumptions and Experimental Controls

- All sorting algorithms receive identical input arrays copied from the same dataset subset.
- The same compiler, optimization flags and numeric data type are used across implementations.
- Timing excludes dataset download and file I/O unless explicitly stated.
- The dataset is preprocessed once; benchmark data is reused to ensure fair comparisons.
- Binary Search is applied only after the data has been sorted.
- Selection Sort at very large n is reported as infeasible when its required work exceeds the benchmark time budget.
- Operation counts are implementation-defined; comparisons and data movements should be reported separately.

22. Conclusion

The study confirms the fundamental behavior of brute-force algorithms. For the given 12-element input, Selection Sort produces the ascending sequence 5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93 after 11 passes and performs 66 minimum-selection comparisons. With swaps counted only when the minimum is different from the current position, the input requires 8 effective swaps. Sequential Search finds 68 at position 8 (one-based), requiring 8 comparisons.

As input size increases, the fixed $\Theta(n^2)$ comparison growth of Selection Sort becomes the dominant limitation. For one million elements it requires 499,999,500,000 comparisons, explaining why it is not a practical large-scale sorting method. Sequential Search grows linearly and is simple when data is unsorted, but Binary Search reduces repeated lookup cost to logarithmic time after sorting. Merge Sort and hybrid sorting approaches provide much better scalability than Selection Sort. Therefore, the

experimental design should combine exact operation counts with actual machine measurements and should report infeasible brute-force runs honestly rather than inventing timings.

23. GitHub Submission Checklist

1. Create a GitHub repository, for example: CSA06-DAA-Brute-Force-Assignment.
2. Add this MS Word report (.docx).
3. Add the C++ source file (for example, main.cpp).
4. Add the dataset source information and, if permitted by licensing, the dataset or a download/readme instruction.
5. Add benchmark output CSV/text files and generated graphs.
6. Add a README.md describing compilation, execution, dataset preparation, hardware/software environment and results.
7. Commit and push all files to the repository.
8. Copy the repository URL and paste it into Google Classroom.

Repository link to paste after upload: <https://github.com/yogi8121660800/CSA-0603-192424319/tree/main/CSA06-DAA-Brute-Force-Assignment>.

24. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.
2. Kleinberg, J., & Tardos, É. Algorithm Design. Pearson.
3. Levitin, A. V. Introduction to the Design & Analysis of Algorithms. Pearson.
4. UCI Machine Learning Repository. HIGGS dataset – public benchmark dataset with 11 million records.
5. ISO/IEC C++ Standard Library documentation for std::chrono and standard algorithms.
6. United Nations. Sustainable Development Goal 4: Quality Education.
7. United Nations. Sustainable Development Goal 9: Industry, Innovation and Infrastructure.

Appendix – Sample Output Screenshots

The following console-style output image represents the expected output for the given 12-element input. It can be used as the output screenshot in the assignment.

Selection Sort

Input : 42 17 8 93 26 55 11 68 34 5 79 21
Pass 1: 5 17 8 93 26 55 11 68 34 42 79 21
Pass 2: 5 8 17 93 26 55 11 68 34 42 79 21
Pass 3: 5 8 11 93 26 55 17 68 34 42 79 21
Pass 4: 5 8 11 17 26 55 93 68 34 42 79 21
Pass 5: 5 8 11 17 21 55 93 68 34 42 79 26
Pass 6: 5 8 11 17 21 26 93 68 34 42 79 55
Pass 7: 5 8 11 17 21 26 34 68 93 42 79 55
Pass 8: 5 8 11 17 21 26 34 42 93 68 79 55
Pass 9: 5 8 11 17 21 26 34 42 55 68 79 93
Pass 10: 5 8 11 17 21 26 34 42 55 68 79 93
Pass 11: 5 8 11 17 21 26 34 42 55 68 79 93

Sorted : 5 8 11 17 21 26 34 42 55 68 79 93
Comparisons = 66
Effective Swaps = 8

Sequential Search

Key = 68
Comparison 1: A[1] = 42 -> NOT FOUND
Comparison 2: A[2] = 17 -> NOT FOUND
Comparison 3: A[3] = 8 -> NOT FOUND
Comparison 4: A[4] = 93 -> NOT FOUND
Comparison 5: A[5] = 26 -> NOT FOUND
Comparison 6: A[6] = 55 -> NOT FOUND
Comparison 7: A[7] = 11 -> NOT FOUND
Comparison 8: A[8] = 68 -> FOUND
Result: 68 found at position 8 (index 7)
Search Comparisons = 8

Figure: Selection Sort and Sequential Search output for the given problem.

Appendix A – Expected Answers for the Given 12-Element Problem

Question	Answer
Sorted sequence	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93
Selection comparisons	66
Effective Selection swaps	8
Key searched	68
Key position (1-based)	8
Key index (0-based)	7
Sequential comparisons to find 68	8

Selection Sort complexity	$\Theta(n^2)$, $O(n^2)$, $\Omega(n^2)$
Sequential Search complexity	Best $\Theta(1)$; average $\Theta(n)$; worst $\Theta(n)$

Appendix B – Viva / Discussion Points

- Why does Selection Sort always make $n(n-1)/2$ comparisons?
- Why is Selection Sort $\Theta(n^2)$ even when the input is already sorted?
- What is the difference between a comparison and a swap/data movement?
- Why does Sequential Search require 8 comparisons for key 68?
- When can Binary Search be used?
- Why is Merge Sort preferred for large datasets?
- Why can a hybrid algorithm outperform a pure algorithm in practice?
- Why should actual execution times not be fabricated?